

[← Go Back](#)

Hardware

Portenta X8 ▼

Tutorials

- 01. Portenta X8 User Manual
- 02. Portenta X8 Fundamentals
- 03. Uploading Sketches to the M4 Core on Arduino Portenta X8
- 04. Data Exchange Between Python® on Linux & Arduino Sketch**
- 05. Managing Containers with Docker on Portenta X8
- 06. Using FoundriesFactory® Waves Fleet Management
- 07. Deploy a Custom Container with Portenta X8 Manager
- 08. How To Build a Custom Image for Your Portenta X8
- 09. How To Update Your Portenta X8
- 10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana
- 11. Output WebGL Content on a Screen
- 12. Multi-Protocol Gateway With Portenta X8 & Max Carrier
- 13. Running WordPress & Database Containers on Portenta X8
- 14. Portenta X8 Firmware

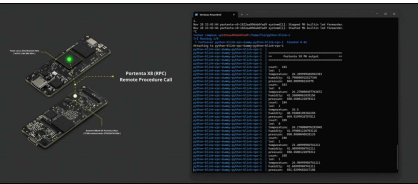
Home / Hardware / Portenta X8 / **04. Data Exchange Between Python® on Linux & Arduino Sketch**

04. Data Exchange Between Python® on Linux & Arduino Sketch

This tutorial will show you how to run a Python® application that exchanges data with an Arduino Sketch.

Author · Sebastian Romero · Last revision · 11/29/2024

Overview



The container infrastructure provided by Arduino contains a pre-built Python® image that you can use to run Python® applications on the Portenta X8. In this tutorial, we are going to build a container based on a provided one.

While all the peripherals are accessible from the iMX8 processor running the Linux environment, it can be useful to let the onboard microcontroller take care of certain peripheral handling and exchange only the required data between the microcontroller and the Python® application.

You will be guided on how to

ON THIS PAGE

Overview

- Goals +
- Remote Procedure Call - RPC
- Python® on the X8 +
- The Arduino Sketch +
- The Python® Application
- Conclusion +

yourself with the foundational elements of the Portenta X8 and its infrastructure by reading [fundamentals of the Portenta X8](#) if you have not already done so.

Goals

Learn how the RPC mechanism on the X8 works

Learn how to exchange sensor data between Linux and an Arduino sketch

Learn how to modify a container and run it

Learn how to use commands to debug the container and service infrastructure

Required Hardware and Software

[Portenta X8](#)

[Portenta breakout](#)

Any sensor (in this example, we will use an [BME680](#) I²C module)

[USB-C® cable \(USB-C® to USB-A cable\)](#)

[Arduino IDE 1.8.10+](#), [Arduino IDE 2](#), or [Arduino Cloud Editor](#)

Remote Procedure Call - RPC

The two processors within the Portenta X8 require a communication mechanism to exchange data, known as an **RPC (Remote Procedure Call)**.

RPC allows a program to trigger a `procedure` or `function` on another computer over a network rather than locally. It lets a program execute procedures **remotely**, with the details of network communication hidden to maintain transparency, making the remote call appear similar to a local one.

It is particularly useful for distributed computing in a client-server model. The `procedure call` behaves as a `request` from the client, and the `return value` serves as the server's `response`. This model uses multiple computers connected over a network (often the Internet) to solve large computational tasks.

While RPC aims to closely replicate local procedure calls, complete equivalence is not possible due to network communication challenges, which can introduce communication failures. To manage these issues, different RPC mechanisms adopt distinct semantics:

`At most once` **semantics** ensures that a remote call may fail but will not be run multiple times.

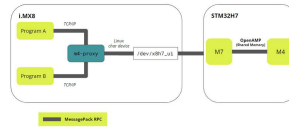
`At least once` **semantics** guarantees that the call is made at least once, even if it results in multiple activations.

The Portenta X8 uses **MessagePack-RPC** for its communication (see the [library repository](#) for details). *MessagePack-RPC* relies on *MessagePack* as the serialization protocol, encoding data in *MsgPack*

SPI

Linux Char Device

TCP/IP



Linux Arduino RPC

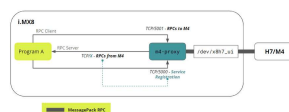
In the image above, the **M7 core** of the **STM32H7** manages communication between the Linux and Arduino environments. If an Arduino sketch runs on the **M4 core**, the **M7 core** acts as an intermediary, handling data requests between the M4 core and the Linux environment. Due to this setup, traditional dual-core processing is not supported on the Portenta X8.

On the Linux side, a service called `m4-proxy` handles data transfer between Linux and Arduino.

The communication process works as follows:

A program registers as the RPC server on port X, listing the procedures available for the M4 to call.

`m4-proxy` then forwards the calls from the M4 to the appropriate program/port.



RPC M4 proxy

Python® on the X8

Python® is a modern and powerful scripting language used for a wide range of applications. In this tutorial, we only read sensor data from an Arduino sketch, but you could extend the example and process the data further.

Communication Between Linux and Arduino Sketches

The Python® script will run on the Linux side and therefore on the iMX8 processor. The Arduino sketch, on the other hand, will run on the STM32H747 microcontroller. It allows for real-time processing on the Arduino side while running a fully-fledged operating system on iMX8.

However, the two processors need a communication mechanism to exchange data with one another.

RPC (Remote Procedure Call) is the communication mechanism for this task. To establish communication, the M7 core on the STM32H747 microcontroller is used to hand over any data/request to the M4 core. That means your Arduino sketch will solely run on the M4 core. Dual-core processing on the Arduino side is currently not supported.

On the Linux side, there is a service that takes care of sending data between the two worlds. It is called `m4-proxy`.

You can check if the service is running by logging into the X8 via `adb shell` and then executing the

```
1 sudo journalctl -fu m4-p
```

If the service has stopped unexpectedly, you can restart it with the following command:

```
1 sudo systemctl restart m
```

The Arduino Sketch

The Arduino sketch to read sensor data doesn't look much different from an ordinary sketch. The only difference is that we expose the sensor data via RPC.

```
1 RPC.bind("temperature",
2 RPC.bind("humidity", []{
3 RPC.bind("pressure", []{
4 RPC.bind("gas", []{ retu
5 RPC.bind("altitude", []{
```

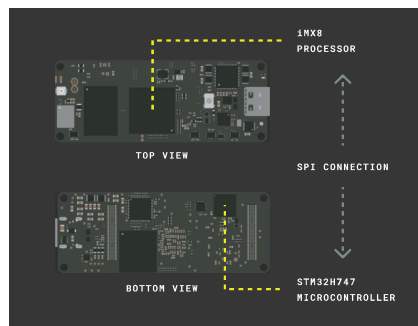
Two additional header files need to be included to enable the RPC mechanism on Portenta X8:

```
1 #include <RPC.h>
2 #include <SerialRPC.h>
```

The `RPC.bind()` method makes the data available via the specified name e.g. "temperature". In our example, an anonymous function is

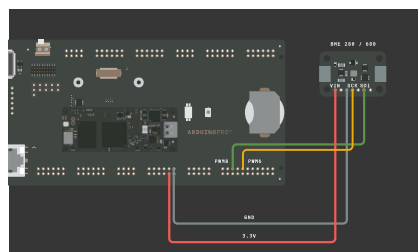
corresponding sensor property whenever requested.

Alternatively, you could bind the name to an existing, named function instead. The data can then easily be requested using that name (e.g. "humidity") by querying the `m4-proxy` service. Once data is requested, it is packaged as a message and sent over SPI to the iMX8.



You can find the sketch in the software package [here](#). You may need to change the sketch depending on the choice of the sensor to read from. If you're using an I²C sensor, you can connect SCL to **PWM6** and SDA to **PWM8** on the Portenta breakout.

That is because the labeled I²C pins on the Portenta Breakout are only available on the Linux side. If you are using an analog sensor, you can connect it to any analog pin. Please refer to the pinout diagram on the Portenta Breakout [documentation page](#).



Make sure you have installed the **Arduino Mbed OS Portenta Boards** core and upload the sketch to the X8 in the Arduino IDE or via Arduino CLI.

Debugging the Arduino Sketch

To check if the Arduino sketch is working correctly, you may want to read the messages from the `Serial.println` statements. You cannot currently read them directly in the serial monitor of the Arduino IDE. Instead, you can use a simple service called `py-serialrpc`, which listens for those messages and prints them to the console.

This service needs to run on the Linux side of the X8. You can get the files [here](#). The compressed file will have every file needed to build a container as the docker compose app. From the command prompt of your local machine, navigate to the adb tool folder and upload the files to the X8 with command:

```
1 adb push <local director
```

Log into the X8 shell with `adb shell` and navigate into the `serialrpc` folder. Build the container using

```
1 docker build . -t py-ser
```


The `-t` flag assigns a tag to the container. Then run the container by executing `cd ..` and then:

```
1 docker compose up -d
```

The `-d` flag detaches the container so it runs in the background. Note that this will run the docker compose app and have the container built persistently across reboots by registering it as a systemd service.

To stop the container, run:

```
1 docker compose stop
```

Check if the container is running by executing:

```
1 docker ps
```

You can then access the log of its service at any time by using following command from the **same directory**:

```
1 docker compose logs -f -
```

If you do not wish to run the container in the background, skip the `-d` flag, you will get the console output directly in the

is running, you will see the messages being sent from the M4.

The Python® Application

The Python® application requests the sensor data from the M4 over RPC and unpacks the message.

Data can be requested by calling the function exposed over RPC on the M4 e.g.:

```
1 m4_proxy_address = 'm4-p
2 m4_proxy_port = 5001
3 rpc_address = RpcAddress
4 rpc_client = RpcClient(r
5 temperature = rpc_client
```

The complete Python® application files are in the same package as the Arduino sketch (see above). Like in the previous step, upload the `python-sensor-rpc` folder to the Portenta X8 via:

```
1 adb push <local director
```

Log into the X8 via `adb shell`. Then navigate into the `python-sensor-rpc` folder and execute:

```
1 docker build . -t python
```

When it has finished, you can run the container with:

```
1 docker compose up
```

After a few seconds, you should see the output from the Python application featuring the sensor readings on the M4 that exchanges through the RPC mechanism. The output should look similar to the following:

```
1 python-sensor-rpc_1 | =  
2 python-sensor-rpc_1 | =  
3 python-sensor-rpc_1 | =  
4 python-sensor-rpc_1 |  
5 python-sensor-rpc_1 | T  
6 python-sensor-rpc_1 | H  
7 python-sensor-rpc_1 | P  
8 python-sensor-rpc_1 | G  
9 python-sensor-rpc_1 | A
```

Whenever you change anything in the Python® script on your computer, you will have to resync and push the new script to the Portenta X8 and rebuild the container. Following command sequence will help you to do this process:

```
1 # On your computer  
2 adb push python-sensor-r
```

```
1 # On the Portenta X8
2 docker build . -t python
```

```
1 # On the Portenta X8
2 docker compose up
```

Alternatively, you could modify the files directly on the X8 using an editor such as **VIM**, so you do not need to upload the files every time. Rebuilding the container will be necessary in any case though.

If you wonder how to specify the Python® script that is executed when running a container, have a look at the `Dockerfile` file. There you will find the `ENTRYPOINT` command that takes multiple arguments. In our example:

```
1 ENTRYPOINT ["python3", "
```

Conclusion

In this tutorial, you learned how to use the docker infrastructure to build a container that runs a Python® application. You have also learned how to use the RPC mechanism to exchange data between the microcontroller and the iMX8, which runs the Linux

Next Steps

You may further process the data you receive from the Arduino sketch and, e.g., upload it to a Cloud service or similar.

Familiarize yourself with Docker commands to adjust the docker configuration to your needs.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

